VesselOps 프로그램 설계서 (기능명세서)

항목	내용
문서명	선박 기관 원격 상태감시 플랫폼 프로그램 설계서
버전	v1.1
작성일	2026-10-12
작성자	최승렬
선행문서	`01-requirements.md` (v1.3), `02-architecture.md` (v1.2), `03-interface-spec.md` (v1.2), `04-database-design.md` (v1.1)
관련문서	`00-glossary.md` (용어 정의), `08-test.md` (검증 결과)

목차

1. 개요
2. 공통 설계
3. PGM-01 · 선단 시뮬레이터
4. PGM-02 · Edge 수집기
5. PGM-03 · 육상 수집기
6. PGM-04 · 업무 API
7. PGM-05 · 화면
8. 설정 항목 일람
9. 구현 범위와 차기 과제
10. 변경 이력

1. 개요

본 문서는 **프로그램이 몇 개이고, 각각 어떤 모듈로 나뉘며, 어떤 순서로 처리하는가**를 정의한다. 프로그램 사이의 규격은 `03-interface-spec.md`, 저장 구조는 `04-database-design.md` 가 다룬다.

1.1 프로그램 목록

ID	프로그램	런타임	역할	소스	관련 요구사항
PGM-01	선단 시뮬레이터 `sim-fleet`	Node.js 22	실선 AMS 를 대신하는 OPC UA 서버. 기관 물리모델과 고장 주입	`simulator/`	범위 2.2, IF-01, IF-06
PGM-02	Edge 수집기 `edge-gw`	Node.js 22	OPC UA 수집 → Deadband·적응형 전송·단절 버퍼 → MQTT 발행	`edge-gw/`	FR-101 ~ FR-110

ID	프로그램	런타임	역할	소스	관련 요구사항
PGM-03	육상 수집기 `ingest`	Node.js 22	MQTT 수신 → 검증 → 배치 적재 → WebSocket 푸시	`ingest/`	FR-151 ~ FR-156
PGM-04	업무 API `api`	Java 21 / Spring Boot 3.5	조회 API, 알람 룰 엔진, 실린더 편차 진단	`api/`	FR-301 ~ FR-305, FR-403, FR-601 ~ FR-605
PGM-05	화면 `web`	정적 HTML·JS / Nginx	선단 현황·편차·추세·알람·시연 패널	`web/`, `deploy/nginx/`	FR-601 ~ FR-605

DB(`db`)와 MQTT 브로커(`broker`)는 기성 제품(TimescaleDB, Mosquitto)을 설정만 하여 사용한다.

1.2 처리 흐름 한눈에 보기



1.3 Node.js 와 Java 의 역할 분담

구간	언어	이 구간의 성격
수집·전송 (PGM-01·02·03)	Node.js	연결을 오래 유지하며 작은 메시지를 많이 주고받는다. 프로토콜 변환(OPC UA ↔ MQTT ↔ WebSocket)이 중심이다
판정·조회 (PGM-04)	Java	룰·상태 전이·이력처럼 정합성이 중요한 업무 규칙이 중심이다. 정적 타입과 단위 테스트로 규칙을 고정한다

두 구간은 **서로를 직접 호출하지 않는다.** MQTT 브로커와 DB 를 사이에 두므로 한쪽을 재배포해도 다른 쪽은 계속 동작한다 (ADR-001).

---

2. 공통 설계

항목	규칙
설정	전부 환경변수로 주입한다. 코드와 이미지에 접속 정보를 넣지 않는다 (8장)
시각	저장·전송은 UTC ISO8601. 화면에서만 사용자 현지시각으로 바꿔 보여준다
로그	표준출력 한 줄 기록. `시각 [프로그램] 내용` 형식. 수준 기준은 `03-interface-spec.md` 9.1
기동 순서	DB·브로커가 늦게 떠도 실패하지 않고 재시도한다. 컨테이너 기동 순서에 의존하지 않는다

항목	규칙
종료	SIGTERM 수신 시 보류 중인 배치를 적재한 뒤 종료한다 (PGM-03)
컨테이너	루트가 아닌 사용자로 실행한다. 외부 공개 포트는 `web` 의 80 하나뿐이다

---

3. PGM-01 · 선단 시뮬레이터

3.1 모듈 구성

파일	책임
`src/fleet.js`	선박 4척 정의. DB 시드(`db/init/02-seed.sql`)와 IMO 번호·장비코드가 1:1 로 대응한다
`src/engine-model.js`	기관 물리모델. 부하율 하나에서 전 계측값을 파생한다
`src/faults.js`	고장 모델. 주입된 고장을 시간에 따라 진행되는 효과로 환산한다
`src/opcua-server.js`	OPC UA 서버. 주소공간 구성과 변수 갱신
`src/index.js`	진입점. 1초 틱 루프와 시연 제어 API

3.2 틱 처리 (1초 주기)

순서	처리
1	유휴 10분이 지난 수동 고장, 수명 5분이 지난 자동 고장을 정리한다
2	선박별로 운항 프로파일 만료를 확인하고 목표 부하를 다시 뽑는다 (30~60분마다)
3	부하율을 목표 쪽으로 천천히 움직인다 (평균회귀 랜덤워크)
4	고장 효과(`fx`)를 계산한다
5	물리모델로 장비별 계측값을 산출한다
6	추력베어링 온도에 열 관성(시정수 300초)을 적용한다
7	OPC UA 변수의 값과 소스 타임스탬프를 갱신한다

3.3 물리모델

값을 태그마다 따로 난수로 만들면 "회전수는 낮은데 출력은 높은" 모순된 데이터가 나온다. **부하율(0~1) 하나가 상태를 결정**하고 나머지는 전부 여기서 파생한다.

계측값	관계식	근거
회전수	정격 rpm × ∛부하	프로펠러 법칙 (출력 ∝ 회전수³)
연료소비율 SFOC	최소값 + 95 × (부하 − 0.75)²	저속 2행정 기관은 75% 부하 부근이 최적인 U자 곡선
연료유량	정격출력 × 부하 × SFOC ÷ 1000	kg/h
소기압력	최대소기압 × 부하^1.4	과급기 특성
배기온도 평균	295 + 170×부하 − 65×부하²	고부하로 갈수록 상승폭이 둔해진다

계측값	관계식	근거
실린더별 배기온도	평균 + 실린더 고유 오프셋 + 잡음 ±3℃	개체차 모사
추력베어링 온도	40 + 16×부하, 300초 지연	열용량이 커서 부하를 늦게 따라간다. 전부하에서도 60℃ 미만

> **열 관성을 넣은 이유.** 지연이 없으면 정상적인 부하 변경(예: 35% → 82%)만으로 베어링 온도가 분당 10℃ 이상 변해 변화율 알람(FR-303)이 오작동한다. 시험 중 실제로 발생했다.

3.4 고장 모델

고장은 값을 한 번에 바꾸지 않고 **시간에 따라 진행**시킨다. 그래야 dwell 판정과 변화율 알람이 의미 있게 동작한다.

종류	효과	진행	유발 알람
`CYL_EXH_DEVIATION`	지정 실린더 배기온도 +75℃ (기본)	120초에 걸쳐 상승	편차 HI (50℃, 30초 지속)
`LO_PRESSURE_DROP`	윤활유 압력 −2.1 bar (3.9 → 1.8)	90초에 걸쳐 하강	변화율 WARN → 임계값 LO (2.0 bar)
`THRUST_BRG_HEATING`	추력베어링 온도 분당 +8℃, 최대 +50℃	선형 상승	변화율 WARN → 임계값 HI → HIHI
`TC_FOULING`	소기압 −15%, 배기온도 +25℃, 연료 +5%	60초에 걸쳐 진행	성능 저하 (알람 없음)

> 편차 고장의 기본 크기가 60℃ 가 아니라 75℃ 인 이유: 한 실린더가 오르면 **평균도 함께 오른다.** 7기통에서 한 실린더를 75℃ 올리면 평균이 약 11℃ 올라 편차는 약 64℃ 가 된다.

3.5 시연 제어 API

`03-interface-spec.md` 8장 참조. 정박 중인 선박(주기관 정지)에 대한 고장 주입은 `409 ENGINE_STOPPED` 로 거절한다.

---

4. PGM-02 · Edge 수집기

4.1 모듈 구성

파일	책임
`src/config.js`	환경변수, 태그별 Deadband 표, 전송 모드 전환 기준
`src/opcua-collector.js`	OPC UA 접속·주소공간 탐색·구독(실패 시 주기 읽기)·품질코드 변환
`src/vessel-edge.js`	선박 1척분 전송 로직. 링버퍼, 모드 판정, Deadband, 요약, 우선순위 큐, MQTT
`src/index.js`	진입점. 1초 틱과 링크 제어 API

시연 환경에서는 프로세스 하나가 4척분을 함께 구동한다. 선박별 로직과 MQTT 접속은 `VesselEdge` 객체로 분리돼 있어, 실선에서는 1척분만 띄우면 된다.

4.2 기동 절차

순서	처리
1	OPC UA 서버에 익명·읽기 전용으로 접속한다. 실패하면 1초 → 최대 30초 간격으로 재시도한다
2	`Vessels` 아래 3단계(선박 → 장비 → 태그)를 탐색해 변수 노드 목록을 만든다
3	전 노드를 한 번 읽어 초기값을 채운다 (구독은 값이 변할 때만 통지하므로)
4	구독을 만든다 (게시 주기 1초, 큐 10). 실패하면 1초 주기 읽기로 대체한다
5	선박별로 MQTT 에 접속한다 (Client ID `edge-{imo}`, 세션 보존, 유언 메시지 등록)

> 태그 목록을 설정 파일로 두지 않고 **탐색으로 구성**한다. 실선에서도 AMS 제조사마다 NodeId 형식이 달라, 문서보다 탐색 결과가 정확하기 때문이다.

4.3 틱 처리 (선박별, 1초 주기)

순서	처리	요구사항
1	현재 표본을 만든다. OPC UA 가 끊겨 있으면 품질을 Bad(0)로 내린다	FR-105
2	표본을 Pre-trigger 링버퍼에 넣는다 (전송 모드와 무관, 600초 보관)	FR-109
3	적응형이 켜져 있으면 전송 모드를 판정한다 (4.5)	FR-108
4	원본 모드: Deadband 를 통과한 값만 장비별로 묶어 P1 큐에 넣는다	FR-103
4'	요약 모드: 통계를 누적하고, 분이 바뀌면 직전 1분 요약을 P2 큐에 넣는다	FR-108
5	큐를 우선순위 순으로 비운다 (틱당 최대 60건)	FR-110

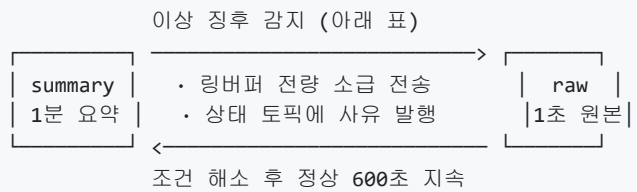
> **표본은 한 초에 정확히 한 번 뜬다.** 매초 0.5초 지점에 틱을 예약하고, 마지막으로 찍은 초를 기억해 같은 초에 두 번 찍지 않는다. 표본 시각은 초 단위로 내려 적으므로, 틱이 밀리거나 일찍 깨면 같은 `(태그, 시각)` 이 두 번 나가거나 한 초가 비게 된다. 둘 다 시험에서 실제로 나왔다 (`08-test.md` 결함 5·6).

4.4 Deadband 판정

다음 중 하나라도 참이면 전송한다.

조건	의미
처음 보내는 태그	초기값
품질코드가 바뀜	센서 상태 변화는 값보다 중요하다
|현재값 − 마지막 전송값| ≥ 태그별 Deadband	의미 있는 변화
마지막 전송 후 60초 경과	Heartbeat. "살아 있고 값이 그대로"임을 알린다

4.5 전송 모드 상태 전이



사유 코드	판정	기준
`DEVIATION`	실린더 배기온도의 평균 대비 최대 편차	30℃ 이상 (육상 알람 50℃ 의 60%)
`THRESHOLD_NEAR`	추력베어링 온도 / 냉각수 출구온도 / 윤활유 압력	60℃ 이상 / 86℃ 이상 / 2.5 bar 이하
`RATE`	60초 전 링버퍼 값과의 차이	추력베어링 4℃, 윤활유 압력 0.4 bar
`QUALITY`	같은 태그의 품질 불량 연속 횟수	5회
`RECOVERED`	위 조건이 모두 해소된 채 경과한 시간	600초

주기관이 정지(5 rpm 미만)해 있으면 `DEVIATION` · `THRESHOLD_NEAR` · `RATE` 는 판정하지 않는다.

> 기준을 **육상 알람보다 낮게** 둔다. 불필요하게 원본 모드로 바뀌는 비용(일시적 대역 증가)이, 늦게 바뀌어 진단 데이터를 잃는 비용보다 작다. 그리고 판정이 다소 늦어도 그 사이 구간은 이미 링버퍼에 들어 있다.

4.6 송신 큐와 단절 처리

등급	내용
P0	상태 메시지 (접속, 전송 모드 전환)
P1	1초 원본, Pre-trigger 소급분
P2	1분 요약

상황	동작
정상	큐에 넣은 틱에 바로 발행한다 (QoS 1)
링크 단절	발행하지 않고 큐에 쌓는다. 이때 넣은 메시지는 `buffered: true` 로 표시한다
상한 초과	큐의 계측값이 100,000건을 넘으면 P2 → P1 순으로 오래된 것부터 버린다
복구	큐가 들어간 순서대로(=시간순) 나간다. 새 표본은 뒤에 붙으므로 순서가 뒤집히지 않는다
순번	`seq` 는 큐에 넣을 때가 아니라 **발행할 때** 매긴다. 수신 측이 누락을 탐지하는 기준이다

시연용 링크 단절(`POST /sim/link`)은 정상 종료 절차 없이 소켓을 끊는다. 그래야 브로커가 유언 메시지를 발행해, 화면이 통신 단절을 실제와 같은 경로로 인지한다.

---

5. PGM-03 · 육상 수집기

5.1 모듈 구성

파일	책임
`src/validate.js`	페이로드 검증 규칙. 순수 함수
`src/index.js`	MQTT 수신, 태그 캐시, 배치 적재, WebSocket, 상태 처리

5.2 계측값 메시지 처리

순서	처리	실패 시
1	JSON 파싱	`SCHEMA_INVALID` 로 격리
2	헤더 검증 (`imo` , `eq` , `mode` , `values` 1~200건, 요약이면 `period`)	메시지 전체를 `SCHEMA_INVALID` 로 격리
3	값마다 검증 (필드 타입, 품질코드, 시각 ±24시간, 유한한 수)	해당 값만 사유별로 묶어 격리
4	`(imo, 장비, 태그코드)` → `tag_id` 를 메모리 캐시에서 찾는다	`UNKNOWN_TAG` 로 격리. 자동 등록하지 않는다
5	원본은 `telemetry` , 요약은 `telemetry_summary` 버퍼에 넣는다	—
6	소급 전송분이 아니면 WebSocket 으로 민다	—
7	버퍼가 200건 이상이면 즉시, 아니면 1초 타이머로 적재한다	5.3

한 메시지 안에서 **정상인 값은 적재하고 불량인 값만 격리**한다. 값 하나 때문에 나머지를 버리지 않는다.

5.3 배치 적재

항목	설계
방식	컬럼별 배열을 `unnest()` 로 풀어 넣는 다중 행 INSERT. 200행 단위
중복	`ON CONFLICT (tag_id, ts) DO NOTHING` . 영향받은 행 수로 중복 건수를 센다
DB 장애	버퍼를 버리지 않고 다음 주기에 다시 시도한다. 보류 상한 10,000건 초과분은 오래된 것부터 폐기한다
태그 캐시	기동 시 적재, 5분마다 갱신. 값마다 조회하면 초당 142회 질의가 발생한다
계수	`received` · `inserted` · `duplicated` · `rejected` · `held` · `discarded` 를 `/health` 로 노출한다

5.4 상태 메시지 처리

수신 내용	처리
`online` 변경	메모리의 선박 상태를 갱신하고 구독 중인 브라우저에 `vesselStatus` 를 민다
`mode` + `reason` 포함	`edge_mode_event` 에 전환 이력을 넣는다. 같은 `(선박, 발생시각)` 이 이미 있으면 넣지 않는다

상태 토픽은 Retain 이라 수집기를 재기동하면 마지막 메시지가 다시 들어온다. 중복 방지 조건은 이 때문이다.

5.5 WebSocket

항목	설계
구독 단위	선박(IMO). `*` 는 전 선박
전송 대상	해당 선박을 구독한 연결에만 보낸다
연결 정리	30초마다 ping. 응답이 없으면 끊는다

---

6. PGM-04 · 업무 API

6.1 패키지 구조

```
com.vesselops
├── VesselOpsApplication        진입점. 스케줄링 활성화
├── common
│   ├── ApiException            업무 오류 (HTTP 상태 + 오류 코드)
│   └── GlobalExceptionHandler  오류 응답 형식 통일
├── telemetry
│   ├── LatestValue             계측점 정의 + 최신값
│   └── LatestValueService      전 계측점 최신값 스냅샷 (0.8초 캐시)
├── diagnosis
│   └── CylinderDeviation       실린더 편차 계산. 순수 함수
├── alarm
│   ├── AlarmRule               룰 모델
│   ├── AlarmRepository         룰·알람 이력 SQL
│   ├── AlarmEngine             룰 평가와 상태 전이 (1초 주기)
│   └── AlarmController         알람 목록·확인
└── vessel
    ├── VesselDtos              응답 모델
    ├── VesselService           선단·상세·편차·추세 조회
    └── VesselController        조회 API
```

설계 선택	이유
JPA 대신 `JdbcClient`	이 시스템의 조회는 `time_bucket`, `LATERAL`, 부분 유니크 인덱스 `ON CONFLICT` 처럼 SQL 자체가 설계다. ORM 으로 감싸면 오히려 가려진다 (ADR-008)
편차 계산을 순수 함수로 분리	핵심 진단 로직을 DB·프레임워크 없이 단위 테스트한다 (`CylinderDeviationTest` , 5건)
최신값 스냅샷을 한 곳에서	알람 엔진과 조회 API 가 같은 스냅샷을 쓴다. 화면과 알람 판정이 서로 다른 값을 보지 않는다

6.2 최신값 스냅샷

계측점마다 최근 10분 안의 가장 최근 값을 한 번의 질의로 가져온다 (`LEFT JOIN LATERAL … LIMIT 1`). 원본이 있으면 원본을, 요약 모드 구간이면 1분 요약의 평균을 최신값으로 쓴다. `uq_telemetry (tag_id, ts DESC)` 인덱스를 태그당 한 번 타므로 142개 계측점 기준 수 밀리초다.

6.3 알람 룰 엔진

6.3.1 평가 절차 (1초 주기)

순서	처리
1	활성 룰을 읽는다 (60초 캐시)
2	최신값 스냅샷에서 **판정 가능한 값**만 고른다 — 품질 Good(192)이고 90초 이내
3	룰마다 판정한다 (6.3.2). 판정할 데이터가 없으면 상태를 그대로 둔다
4	판정 결과로 상태를 전이한다 (6.3.3)

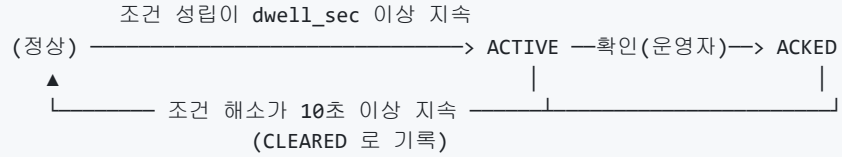
> **데이터가 없으면 건드리지 않는다.** 통신이 끊겨 값이 안 들어오는 것을 "정상 복귀"로 오인해 알람을 해제하면 안 된다. 열려 있던 알람은 통신이 복구돼 실제 값으로 판정될 때까지 유지된다.

6.3.2 룰 종류별 판정

룰	판정식	비고
`THRESHOLD`	HI·HIHI: 값 ≥ 기준 / LO·LOLO: 값 ≤ 기준	등급이 방향을 정한다
`DEVIATION`	같은 장비의 실린더 값 평균을 내고, |값 − 평균| 의 최댓값 ≥ 기준	유효 실린더 2개 미만이면 판정 불가. 가장 벌어진 실린더를 알람 대상으로 기록한다
`RATE_OF_CHANGE`	현재값 − (구간 전 값). 기준이 양수면 상승, 음수면 하강을 감시	구간 전 값은 `telemetry` 에서 조회한다

주기관 정지 억제. 주기관 룰은 `ME_RPM` 이 5 미만이면 "정상"으로 판정한다. 정지 중에는 윤활유 압력 등이 당연히 낮으므로, 억제하지 않으면 정박 중인 선박이 항상 알람 상태가 된다.

6.3.3 상태 전이



항목	설계
발생 지연	룰의 `dwell_sec` . 편차 HI 30초, 윤활유 압력 LOLO 2초처럼 위험도에 따라 다르다
해제 지연	10초. 경계값에서 켜졌다 꺼졌다 하는 채터링을 막는다 (FR-304)
중복 방지	`ON CONFLICT (rule_id) WHERE state <> 'CLEARED' DO NOTHING` . 룰당 미해제 알람은 1건
재기동	기동 시 미해제 알람을 읽어 이어받는다. 그러지 않으면 재기동 때마다 해제가 누락된다
확인	`ACTIVE` 일 때만 가능. 확인해도 조건이 해소돼야 해제된다

6.4 API 명세

기본 경로 `/api/v1` . 공통 규약과 오류 형식은 `03-interface-spec.md` 7장.

6.4.1 `GET /vessels` — 선단 현황


```
[
  {
    "imo": "9876543", "name": "STELLA PIONEER", "vesselType": "CONTAINER", "classSociety": "KR",
    "status": "AT_SEA", "sisterGroup": "SP-CLASS", "engineModel": "7S60ME-C10.5", "cylinderCount": 7,
    "online": true, "lastSeenAt": "2026-10-10T08:05:11Z",
    "loadPct": 57.639, "rpm": 73.674, "foFlowKgH": 1519.705, "shaftPowerKw": 8738.363,
    "maxCylinderDeviation": 7.03, "openAlarmCount": 0, "alarmLevel": "NORMAL"
  }
]
```


필드	산출
`online`	마지막 수신이 20초 이내
`maxCylinderDeviation`	주기관 운전 중일 때만 계산. 정지 중이면 없음
`alarmLevel`	미해제 알람 중 최고 등급 — `CRITICAL` (HIHI·LOLO) / `ALARM` (HI·LO) / `WARN` / `NORMAL`

6.4.2 `GET /vessels/{imo}` — 선박 상세

선단 현황과 같은 요약(`vessel`)에, 장비별 계측점 목록(`equipment[].tags[]`)을 더해 돌려준다. 계측점마다 `code` · `name` · `unit` · `cylinderNo` · `normalMin` · `normalMax` · `value` · `quality` · `ts` 를 가진다.

6.4.3 `GET /vessels/{imo}/cylinder-deviation` — 실린더 편차


```
{
  "imo": "9876543", "engineRunning": true,
  "mean": 399.42, "maxDeviation": 65.44, "limitHi": 50.0, "limitHiHi": 80.0,
  "at": "2026-10-10T08:09:33Z",
  "cylinders": [
    { "no": 3, "tag": "ME_EXH_T_CYL03", "value": 390.64, "deviation": -8.78, "level": "NORMAL" },
    { "no": 4, "tag": "ME_EXH_T_CYL04", "value": 464.85, "deviation": 65.44, "level": "ALARM" }
  ]
}
```


`level`	조건
`STOPPED`	주기관 정지
`CRITICAL`	|편차| ≥ HIHI 기준
`ALARM`	|편차| ≥ HI 기준
`WARN`	|편차| ≥ HI 기준의 60% (Edge 가 원본 모드로 바꾸는 기준과 같다)
`NORMAL`	그 외

`limitHi` · `limitHiHi` 는 `alarm_rule` 에 등록된 값을 그대로 내려준다. 화면의 경고선과 알람 기준이 어긋나지 않는다.

6.4.4 `GET /vessels/{imo}/trend` — 추세

파라미터	필수	설명
`tags`	✅	태그코드 1~12개, 쉼표 구분
`minutes`	—	조회 기간. 기본 30, 범위 1~1440

기간이 길어져도 응답 크기가 일정하도록 구간 폭을 `기간(초) ÷ 240` 으로 잡아 `time_bucket` 평균을 낸다.

```
{
  "imo": "9876543", "from": "2026-10-10T07:55:00Z", "to": "2026-10-10T08:10:00Z", "bucketSec": 3,
  "series": [
    { "tag": "ME_EXH_T_CYL04", "name": "4번 실린더 배기온도", "unit": "degC",
      "points": [[1791619500000, 390.87], [1791619503000, 391.12]] }
  ]
}
```

6.4.5 `GET /alarms` — 알람 목록

파라미터	설명
`imo`	선박 필터
`state`	`OPEN` (미해제) / `ACTIVE` / `ACKED` / `CLEARED` . 생략하면 전체
`page` , `size`	0부터. `size` 는 1~200, 기본 50

발생 시각 내림차순. 응답은 `{ content[], page, size, total }` .

6.4.6 `POST /alarms/{id}/ack` — 알람 확인

결과	응답
성공	200, 갱신된 알람
알람 없음	404 `ALARM_NOT_FOUND`
이미 확인·해제됨	409 `ALARM_NOT_ACTIVE`

6.4.7 오류 코드

코드	HTTP	상황
`VESSEL_NOT_FOUND`	404	없는 IMO 번호
`ALARM_NOT_FOUND`	404	없는 알람
`ALARM_NOT_ACTIVE`	409	확인할 수 없는 상태
`TAGS_INVALID` / `RANGE_INVALID`	400	추세 조회 파라미터 오류
`STATE_INVALID` / `PAGE_INVALID` / `PARAM_INVALID`	400	목록 조회 파라미터 오류
`INTERNAL_ERROR`	500	처리되지 않은 예외. 상세는 서버 로그에만 남긴다

---

7. PGM-05 · 화면

단일 페이지. 빌드 과정 없이 정적 파일 3개(`index.html` , `style.css` , `app.js`)로 구성하고 Nginx 가 제공한다.

영역	데이터 출처	갱신
선단 현황 카드	`GET /vessels`	3초
주요 계측값·전체 계측값	`GET /vessels/{imo}` + WebSocket	수신 즉시 (화면 갱신은 프레임당 1회로 묶는다)

영역	데이터 출처	갱신
실린더 편차 차트	`GET /vessels/{imo}/cylinder-deviation`	2초
추세 차트	`GET /vessels/{imo}/trend`	5초
알람 목록	`GET /alarms`	3초
시연 패널	`GET /sim/state` , `GET /sim/link`	3초

설계 선택	이유
편차를 **평균 기준 위아래 막대**로 그린다	절대 온도 막대는 부하가 바뀌면 전체가 함께 움직여 편차가 눈에 띄지 않는다. 평균을 0 으로 두면 벌어진 실린더만 튀어나온다
정지 중인 기관은 범위 이탈로 칠하지 않는다	알람 엔진의 정지 억제와 같은 기준이다
로그인 없이 바로 본다	시연 사이트에 로그인 벽을 두면 평가자가 들어오지 않는다
Nginx 가 `/api` · `/ws` · `/sim` 을 중계한다	브라우저는 한 곳만 본다. 뒤쪽 서비스의 주소와 포트가 밖으로 드러나지 않는다

---

8. 설정 항목 일람

프로그램	환경변수	기본값	설명
공통	`POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD`	`vesselops` / `vesselops` / *(개발용)*	DB 접속. **운영에서는 반드시 바꾼다**
sim-fleet	`SIM_INTERVAL_MS`	1000	틱 주기
sim-fleet	`SIM_AUTO_FAULT`	true	평상시 경미한 이상 자동 발생
sim-fleet	`SIM_IDLE_RESET_SEC`	600	마지막 조작 후 자동 정상 복귀
edge-gw	`OPCUA_ENDPOINT`	`opc.tcp://sim-fleet:4840/vesselops`	실선에서는 AMS 주소로 바꾼다
edge-gw	`COLLECT_MODE`	subscribe	`poll` 이면 주기 읽기
edge-gw	**`ADAPTIVE_ENABLED`**	**false**	시연 프로파일은 false(항상 원본), 실선 프로파일은 true
edge-gw	`RING_SEC` / `RETURN_SEC`	600 / 600	Pre-trigger 보관 / 요약 복귀 대기
edge-gw	`SUMMARY_PERIOD_SEC`	60	요약 구간
ingest	`INGEST_BATCH_SIZE` / `INGEST_FLUSH_MS`	200 / 1000	배치 적재 조건

프로그램	환경변수	기본값	설명
api	`vesselops.alarm.fresh-seconds`	90	판정에 쓰는 값의 유효 시간. Heartbeat(60초)보다 길어야 한다
api	`vesselops.alarm.clear-hold-seconds`	10	해제 지연
api	`vesselops.alarm.engine-running-rpm`	5	주기관 운전 판정 기준
api	`vesselops.online.timeout-seconds`	20	통신 단절 표시 기준
web	`WEB_PORT`	80	외부 공개 포트

9. 구현 범위와 차기 과제

요구사항정의서의 전 항목을 이번 버전에 구현한 것은 아니다. 구현하지 않은 것을 구현한 것처럼 적지 않는다. 항목별 상태는 `10-traceability.md` 가 관리한다.

구분	내용	상태
수집·전송	OPC UA 구독, Deadband, 적응형 전송, Pre-trigger, 우선순위 큐, 단절 버퍼	✅ 구현·검증
수신·적재	검증·격리, 배치 적재, 중복 제거, WebSocket	✅ 구현·검증
진단·알람	임계값·편차·변화율, dwell, 확인·해제	✅ 구현·검증
화면	선단·편차·추세·알람·시연 패널	✅ 구현
단절 버퍼의 디스크 저장 (FR-104)	현재는 메모리 큐. Edge 프로세스가 재기동되면 버퍼가 사라진다	⏳ 차기
추세의 원본·요약 병합 조회 (`04` 5장)	현재 추세는 원본만 조회한다. 요약 구간은 최신값에만 반영된다	⏳ 차기
본선 발행 알람 토픽 (`03` 4.4)	알람 판정은 육상 API 가 수행한다. 본선 자체 판정·발행은 미구현	⏳ 차기
인증·권한 (FR-501 ~ 503)	`app_user` 테이블만 있다. 조회는 게스트 공개, 알람 확인도 현재 무인증	⏳ 차기
마스터 등록·수정 (FR-201 ~ 204)	시드 SQL 로만 등록한다. 조회만 API 로 제공	⏳ 차기
성능 분석 (FR-401, 402, 404, 405)	실린더 편차 차트(FR-403)만 구현	⏳ 차기
통신 암호화 (NFR-05)	MQTT 구간은 TLS + 선박별 클라이언트 인증서 적용 (`02` ADR-011, `03` 4.1). HTTPS 는 미적용	🔶 부분

10. 변경 이력

버전	일자	내용
v1.0	2026-10-10	최초 작성. 구현 완료된 프로그램 5종의 모듈 구성·처리 절차·API 명세와 차기 과제 정리
v1.1	2026-10-12	통신 암호화 상태 갱신 (MQTT TLS 적용). 접속·푸시 규격의 변경은 `03-interface-spec.md` v1.3 참조